```
#include <Servo.h>
#include <LiquidCrystal.h>

// ── Pin Definitions ──────────────────────────────────────────
#define STEP_PIN     6    // NEMA17 stepper – STEP pulse
#define DIR_PIN      7    // NEMA17 stepper – DIR
#define EN_PIN       5    // NEMA17 stepper – ENABLE (active LOW)

#define SERVO1_PIN   A5   // Servo 1 – moisture sorting gate
#define SERVO2_PIN   A4   // Servo 2 – metal   sorting gate

#define LDR_PIN      A0   // LDR sensor       (analog)
#define MOISTURE_PIN A1   // Moisture sensor  (analog)
#define METAL_PIN    2    // Metal detector   (digital, HIGH = detected)

#define BUZZER_PIN   A2   // Buzzer ← moved here; D8 is now used by LCD

// ── Sensor Thresholds  (adjust to match your hardware) ──────
#define LDR_THRESHOLD       500  // analog BELOW this → object detected
#define MOISTURE_THRESHOLD  500  // analog ABOVE this → moisture detected

// ── Conveyor Step Counts ─────────────────────────────────────
#define STEPS_A   85   // conveyor travel for moisture sort (Command A)
#define STEPS_B   400  // conveyor travel for metal   sort (Command B)
#define STEPS_C   700  // conveyor travel for metal   sort (Command B)
```

```
// ── Stepper Timing ──────────────────────────────────────────
#define STEP_HALF_MS  10  // half-period in ms (10 ms each side = 20 ms/step)

// ── Servo Positions ─────────────────────────────────────────
#define SERVO_HOME   170  // resting / pass-through position
#define SERVO_SORT    10  // divert position

// ── Objects ─────────────────────────────────────────────────
Servo          servo1;
Servo          servo2;
LiquidCrystal   lcd(       13,  12,  11, 10,  9,  8);

// ── State ───────────────────────────────────────────────────
int          stepCount       = 0;     // remaining steps to run
char         activeCmd       = '.';   // current command in progress ('A', 'B', or '.')
bool         servoFire       = false; // flag: conveyor finished, fire servo next
unsigned long lastSensorUpdate = 0;     // millis() timestamp of last sensor display refresh

//
═══════════════════════════════════════════════════════════════════════
══════════

//  HELPERS
//
═══════════════════════════════════════════════════════════════════════
══════════
```

```
// Clear LCD and print two lines ————————————————————————
void lcdPrint(const char* line1, const char* line2) {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(line1);
  lcd.setCursor(0, 1);
  lcd.print(line2);
}

// Single short beep ————————————————————————————————————
void beep() {
  digitalWrite(BUZZER_PIN, HIGH);
  delay(200);
  digitalWrite(BUZZER_PIN, LOW);
}


// Single step pulse ————————————————————————————————————
void doStep() {
  digitalWrite(STEP_PIN, HIGH);
  delay(STEP_HALF_MS);
  digitalWrite(STEP_PIN, LOW);
  delay(STEP_HALF_MS);
}

// Begin a conveyor run (ignored while already running) ——————————
```

```
void startConveyor(char cmd, int steps) {
  if (stepCount == 0 && activeCmd == '.') {
    activeCmd = cmd;
    stepCount = steps;
    servoFire = false;

    // ── Buzzer: single beep on detection ──────────────────────
    beep();

    if (cmd == 'A') {
      lcdPrint("Apple Det.", "Sorting: Gate 1");
      Serial.print("*Apple#");
    } else if (cmd == 'B') {
      lcdPrint("Guava Detected", "Sorting: Gate 2");
      Serial.print("*Guava#");
    }
  else if (cmd == 'C') {
      lcdPrint("Lemon Detected", "Sorting: Gate 2");
      Serial.print("*Lemon#");
    }
    Serial.print("CMD ");
    Serial.print(cmd);
    Serial.print(" - running ");
    Serial.print(steps);
    Serial.println(" steps");
  }
```

```
}

//
═══════════════════════════════════════════════════════════════════════
═══════════
//  SETUP
//
═══════════════════════════════════════════════════════════════════════
═══════════
void setup() {
  pinMode(STEP_PIN,   OUTPUT);
  pinMode(DIR_PIN,    OUTPUT);
  pinMode(EN_PIN,     OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(METAL_PIN,  INPUT);

  digitalWrite(EN_PIN,     LOW);   // enable stepper driver
  digitalWrite(DIR_PIN,    HIGH);  // forward direction
  digitalWrite(BUZZER_PIN, LOW);

  // ── LCD init ──────────────────────────────────────────────
  lcd.begin(16, 2);
  lcdPrint("   CONV.", "  Initializing");

  Serial.begin(9600);

  servo1.attach(SERVO1_PIN);
```

```
  servo2.attach(SERVO2_PIN);
  servo1.write(SERVO_HOME);
  servo2.write(SERVO_HOME);

  lcdPrint(" System  Ready", "  Waiting...  ");
  Serial.println("Conveyor ready. Send A or B, or let sensors trigger.");
}

//
═══════════════════════════════════════════════════════════════════════════
═══════════
//  MAIN LOOP
//
═══════════════════════════════════════════════════════════════════════════
═══════════
void loop() {

  // ── 1. Read sensors ──────────────────────────────────────────────

  // ── 3. Serial command (manual override) ──────────────────────────
  while (Serial.available()) {
    char c = (char)Serial.read();
    if (c == 'A') { startConveyor('A', STEPS_A); }
    if (c == 'B') { startConveyor('B', STEPS_B); }
    if (c == 'C') { startConveyor('C', STEPS_C); }
  }
```

```
// ── 4. Run conveyor stepper ──────────────────────────────────
if (stepCount > 0) {
  stepCount--;
  doStep();
  Serial.print(activeCmd);
  Serial.println(stepCount);

  // Live step count on LCD row 2 while conveyor runs
  lcd.setCursor(0, 1);
  lcd.print("Steps left: ");
  lcd.print(stepCount);
  lcd.print("   ");       // overwrite any leftover digits

  if (stepCount == 0) {
    servoFire = true;
  }
}

// ── 5. Actuate sorting servo after conveyor stops ────────────
if (servoFire) {
  servoFire = false;

  if (activeCmd == 'A') {
    lcdPrint("Apple", "Gate 1 Opening");
    servo1.write(SERVO_SORT);
    delay(2000);
```

```
    lcdPrint("Apple", "Gate 1 Closing");
    servo1.write(SERVO_HOME);
    delay(2000);
   }
   else if (activeCmd == 'B') {
    lcdPrint("Guava", "Gate 2 Opening");
    servo2.write(SERVO_SORT);
    delay(2000);
    lcdPrint("Guava", "Gate 2 Closing");
    servo2.write(SERVO_HOME);
    delay(2000);
   }

   activeCmd = '.';
   lcdPrint(" Sort Complete!", "  Waiting...  ");
   Serial.println("Sort complete. Ready.");
  }
}
```